MATEMÁTICA & SIMULACIÓN — DEEP LEARNING WORKSHOP

Informe de Laboratorio: Visión Artificial y Detección de Objetos en Tiempo Real

Estudiante: Jesus David Camacho

Repositorio GitHub: Intro_VisionArtificial

Librerías Core: OpenCV 4.x, Ultralytics YOLOv8

Entorno de Ejecución: Python 3.10+ (.venv)

Resumen Ejecutivo: En este laboratorio se desarrolló e implementó un sistema integral de visión por computadora en tiempo real utilizando la cámara web local. Se evaluaron dos paradigmas computacionales para el análisis visual: la clasificación tradicional basada en características Haar Cascade y la detección moderna basada en redes neuronales convolucionales profundas de un solo paso mediante el modelo YOLOv8 (You Only Look Once). Adicionalmente, se construyeron aplicaciones prácticas orientadas al control de aforo (contador de personas), detección de dispositivos móviles y anonimización dinámica de rostros para preservación de privacidad.

1. INTRODUCCIÓN Y MARCO TEÓRICO

La visión artificial es una disciplina dentro de la inteligencia artificial orientada a permitir que las computadoras extraigan interpretaciones de alto nivel a partir de imágenes o secuencias de video. Su implementación práctica abarca desde sistemas de seguridad y asistencia en la conducción hasta el control de calidad industrial [1].

1.1 Detección Clásica mediante Clasificadores Haar Cascade

El método de Paul Viola y Michael Jones (2001) utiliza funciones tipo Haar que evalúan diferencias de intensidad en regiones adyacentes de la imagen. Mediante una representación matemática denominada *Imagen Integral*, el costo de calcular sumas de píxeles se vuelve constante O(1). Para optimizar la inferencia, los clasificadores se organizan en una arquitectura en cascada de rechazo (*Cascade Architecture*), donde las subventanas que no contienen la estructura deseada son descartadas en las primeras etapas, reduciendo drásticamente la carga computacional [1].

1.2 Detección Moderna mediante YOLOv8 (Ultralytics)

A diferencia de los enfoques tradicionales de dos etapas (como R-CNN), la arquitectura YOLO enmarca el problema de detección como una sola regresión de mapeo directo desde los píxeles de la imagen hacia las coordenadas de las cajas delimitadoras (*bounding boxes*) y las probabilidades de clase [2]. YOLOv8 adopta una red backbone libre de anclas (*anchor-free*), lo que permite una predicción más precisa y rápida, óptima para dispositivos con recursos de hardware convencionales.

2. ESTRUCTURA DEL PROYECTO Y ENTORNO VIRTUAL

Para mantener la reproducibilidad e independencia de dependencias, la práctica se estructuró en un repositorio dedicado registrado en GitHub bajo la arquitectura de entorno virtual de Python (`.venv`).

Archivo / Script	Propósito y Componentes Clave	Dependencias Principales
`01_webcam_basica.py`	Verificación de captura de video streaming y buffer de fotogramas.	`cv2.VideoCapture`
`02_detector_rostros.py`	Detección de rostros frontales mediante Haar Cascades en escala de grises.	`CascadeClassifier`
`03_detector_objetos_yolo.py`	Inferencia multiclase en tiempo real con modelo preentrenado COCO.	`ultralytics.YOLO`
`04_contador_personas.py`	Filtrado de clases específicas para conteo dinámico de aforo (`person`).	`YOLO` , `cv2.putText`
`05_desenfoque_rostros.py`	Filtro dinámico de privacidad aplicando Blur Gaussiano sobre ROI detectada.	`cv2.GaussianBlur`

3. DESARROLLO EXPERIMENTAL Y ANÁLISIS DE CÓDIGO

3.1 Captura Básica de Video (01_webcam_basica.py)

El punto de entrada al procesamiento digital requiere la inicialización del objeto `cv2.VideoCapture(0)`, el cual interactúa con el driver de la cámara web. Se establece un bucle infinito que extrae fotograma a fotograma hasta recibir la instrucción de interrupción por teclado (tecla 'q').

```
import cv2

camara = cv2.VideoCapture(0)
if not camara.isOpened():
    print("Error: No se pudo acceder a la cámara.")
    exit()

while True:
    ret, frame = camara.read()
    if not ret:
        break
    cv2.imshow("Webcam Basica - Q para salir", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

camara.release()
cv2.destroyAllWindows()
```

3.2 Detección de Rostros con Haar Cascade (02_detector_rostros.py)

Para ejecutar la detección, la imagen BGR se transforma a escala de grises mediante `cv2.COLOR_BGR2GRAY`, dado que las características Haar analizan gradientes de luminosidad y no información cromática. El método `detectMultiScale` aplica una escala decreciente (`scaleFactor=1.1`) y exige al menos 5 vecinos de confirmación (`minNeighbors=5`).

```
import cv2

camara = cv2.VideoCapture(0)
ruta_xml = cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
detector = cv2.CascadeClassifier(ruta_xml)

while True:
    ret, frame = camara.read()
    if not ret: break

    gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    rostros = detector.detectMultiScale(gris, scaleFactor=1.1, minNeighbors=5, minSize=
(60, 60))

    for (x, y, w, h) in rostros:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(frame, "Rostro", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255,
0), 2)

    cv2.imshow("Detector Haar Cascade", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"): break

camara.release()
cv2.destroyAllWindows()
```

3.3 Detección Multiclase con YOLOv8 (03_detector_objetos_yolo.py)

Haciendo uso del modelo liviano `yolov8n.pt` (Nano), se realiza la predicción sobre las 80 clases del dataset COCO. Se configura un umbral de confianza `conf=0.5` para descartar detecciones ambiguas.

```
from ultralytics import YOLO

# Carga de pesos livianos preentrenados
modelo = YOLO("yolov8n.pt")

# Inferencia continua sobre la fuente de video 0
modelo.predict(source=0, show=True, conf=0.5)
```

3.4 Aplicaciones Prácticas: Conteo y Anonimización de Rostros

A partir de las cajas delimitadoras obtenidas por los detectores, se estructuraron dos aplicaciones avanzadas:

- **Control de Aforo (Contador de Personas):** Se itera sobre las detecciones de YOLOv8, extrayendo el ID de clase `caja.cls`. Si el nombre mapeado equivale a `"person"`, se incrementa un contador acumulador presentado dinámicamente sobre el video con `cv2.putText`.

- **Filtro de Privacidad Dinámico:** Se extrae la Región de Interés (*ROI*) correspondiente al rectángulo del rostro detectado por Haar Cascade y se aplica un desenfoque Gaussiano profundo con una matriz de kernel de (99, 99):
  `rostro_blur = cv2.GaussianBlur(rostro, (99, 99), 30)`

4. ANÁLISIS COMPARATIVO: HAAR CASCADE VS. YOLOV8

A través de las pruebas experimentales realizadas en el laboratorio, se establecieron las principales diferencias operativas entre ambos enfoques:

Criterio de Evaluación	OpenCV Haar Cascade	Ultralytics YOLOv8 (Nano)
Arquitectura de Fondo	Clasificador de características sumatorias de Haar.	Red Neuronal Convolucional Profunda (CNN).
Diversidad de Clases	Limitado a un objetivo específico por archivo XML.	Detección simultánea de 80 clases COCO.
Robustez a Iluminación	Baja. Sensible a sombras y bajo contraste.	Alta. Mantiene precisión con variaciones cromáticas.
Tolerancia a Oclusión	Deficiente ante rotación de rostro o mascarillas.	Elevada. Identifica objetos parcialmente cubiertos.
Consumo de Recurso CPU	Mínimo. Ideal para microcontroladores simples.	Moderado a Alto. Optimizado con GPU/CUDA.

5. DIAGNÓSTICO DE ERRORES COMUNES Y MITIGACIÓN

- **Fallo de Acceso a Dispositivo (Video Device Busy):** Ocurre cuando otras plataformas (Teams, Zoom) bloquean el canal. Solución: Cierre de procesos o alternar el índice en `cv2.VideoCapture(1)`.
- **Falsos Positivos en Haar Cascade:** La presencia de patrones geométricos similares a ojos/nariz genera cuadros erróneos. Solución: Incrementar `minNeighbors=7` y definir un área mínima razonable con `minSize=(80, 80)`.
- **Latencia en Inferencia YOLO:** Si la tasa de refresco disminuye por limitaciones de CPU, se recomienda redimensionar la captura a 640x480 mediante `camara.set(cv2.CAP_PROP_FRAME_WIDTH, 640)`.

6. CONCLUSIONES Y TRABAJO FUTURO

La práctica permitió contrastar de forma empírica la evolución de la visión por computadora. Mientras que los clasificadores clásicos Haar Cascade mantienen su vigencia para tareas ligeras y específicas en hardware restringido, los modelos de Deep Learning como YOLOv8 ofrecen un desempeño superior en entornos complejos, cambiantes y no controlados.

Como trabajo futuro a partir de este repositorio base, se propone la implementación de seguimiento de objetos en tiempo real (*ByteTrack/DeepSORT*) para evitar re-conteos de personas y el entrenamiento personalizado de YOLOv8 (*Fine-Tuning*) para la detección de equipo de protección personal (EPP) en entornos industriales.

7. REFERENCIAS BIBLIOGRÁFICAS

1. Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. **[Documentación OpenCV]**
2. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788. **[Documentación Ultralytics YOLO]**